

Getting Started

How to get the material, set up Git, GitHub and Python on your laptop, join Classroom 50, and submit every part of the Qubit Workbook. Google Colab is covered as an alternative for the Python work.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

COURSE

25AID432 Mathematical Postulates of
Quantum Mechanics

REPOSITORY

github.com/Amrita-School-of-AI/mpqt-25aid432

CLASSROOM

qts-mpqt on Classroom 50

TIME NEEDED

About one hour, once

OFFERING

Odd Semester 2026-27

Read this first

This guide takes you from a laptop with nothing installed to a submitted piece of work. Do it in one sitting, in order, with your laptop and your college email open.

Two places matter in this course:

PLACE	WHAT IT IS	WHAT YOU DO THERE
The course repository, <code>mpqt-25aid432</code>	Public. Notes, slides, the assignment statement	Read and download
Classroom 50, classroom <code>qts-mpqt</code>	One private repository per piece of graded work, created for you	Put your work in it and submit

Every part of the class assignment (the Qubit Workbook) is submitted through Classroom 50, including the handwritten parts, which you scan to PDF. Submitting uses two small command-line tools, **Git** and the **GitHub CLI**, on your own laptop. This guide installs both and walks you through a practice item before anything is marked.

BY THE END OF THIS GUIDE YOU WILL HAVE

- Git, the GitHub CLI and Python with NumPy installed and checked.
- A GitHub account with an SSH key, linked to Classroom 50.
- Accepted and submitted the A0 warm-up, and read the grader's report on it.
- A clear picture of which files go into each part of the Qubit Workbook.
- A way to do the Python work in Google Colab if you prefer.

Part 1: The course repository

github.com/Amrita-School-of-AI/mpqt-25aid432

Public. No login needed to read or download.

```
mpqt-25aid432/  
├─ handbook/      this guide, and the course handbook (plan, marks, policies)  
├─ notes/         three notes booklets, one per unit  
├─ slides/        the lecture decks, q01 to q08  
├─ assignments/   the Qubit Workbook statement: Parts A, B and C  
└─ exercises/     the Part C starter code and its public tests
```

BOOKLET	COVERS
Unit 1: Complex Numbers	The complex plane, arithmetic, conjugate and modulus, Euler's formula, polar form, phase
Unit 2: Linear Algebra	Vectors and matrices over complex numbers, inner products, bases, projection, Hermitian and unitary matrices, eigenvalues, tensor products
Unit 3: The Postulates of Quantum Mechanics	Bra-ket notation, the qubit, measurement, operations, two qubits and entanglement, observables

To download one PDF, open it on the GitHub page and use the download button at the top right of the file view. To download everything at once, use the green **Code** button, then **Download ZIP**.

NOTE · PRINT THE NOTES, OR KEEP THEM ON A TABLET

The notes are written to be worked through with a pen beside them. Reading them on a phone, one screen at a time, makes it hard to follow a calculation that runs over half a page.

Part 2: Install Git, the GitHub CLI and Python

You need three things:

TOOL	WHAT IT IS FOR
Git	Keeps the history of your files and sends them to GitHub
GitHub CLI (<code>gh</code>)	Talks to GitHub and Classroom 50 from the terminal
Python 3 with NumPy	Checks your hand calculations, and runs Part C

Follow the section for your operating system, then do the check in 2.4.

2.1 Windows

1. **Git.** Download the installer from `git-scm.com/download/win` and run it. The default choice on every screen is fine. This also installs **Git Bash**, a terminal that understands the commands in this guide.
2. **GitHub CLI.** Download the Windows installer (`.msi`) from `cli.github.com` and run it.
3. **Python.** Download Python 3 from `python.org/downloads` . On the first screen of the installer, tick **Add python.exe to PATH**, then click **Install Now**.
4. Close every open terminal. Open **Git Bash** from the Start menu, and install NumPy:

```
py -m pip install numpy
```

From now on, **do everything in Git Bash**, and where this guide writes `python` , you type `py` .

PITFALL · FORGETTING TO TICK ADD TO PATH

The most common Windows problem. The symptom is `py` or `python` not being found. Run the Python installer again, choose **Modify**, and tick the option.

2.2 macOS

Open **Terminal** (Applications, Utilities, Terminal).

1. **Git.** Run `xcode-select --install` and accept the dialog. This installs Git and a few other developer tools.
2. **GitHub CLI.** If you use Homebrew, run `brew install gh` . Otherwise download the macOS package (`gh_..._macOS_universal.pkg`) from the releases page linked on `cli.github.com` and open it.

3. **Python.** Install Python 3 from `python.org/downloads`, then:

```
python3 -m pip install numpy
```

Where this guide writes `python`, you type `python3`.

2.3 Linux (Ubuntu or Debian)

Git, Python and NumPy come from the normal package manager:

```
sudo apt update
sudo apt install git python3 python3-numpy wget
```

The GitHub CLI in the Ubuntu archive is too old for Classroom 50. Install the current one from GitHub's own package repository:

```
sudo mkdir -p -m 755 /etc/apt/keyrings
wget -qO- https://cli.github.com/packages/githubcli-archive-keyring.gpg \
| sudo tee /etc/apt/keyrings/githubcli-archive-keyring.gpg > /dev/null
sudo chmod go+r /etc/apt/keyrings/githubcli-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/githubcli-archive-
keyring.gpg] https://cli.github.com/packages stable main" \
| sudo tee /etc/apt/sources.list.d/github-cli.list > /dev/null
sudo apt update
sudo apt install gh
```

On Fedora use `sudo dnf install gh`; on Arch, `sudo pacman -S github-cli`. Where this guide writes `python`, you type `python3`.

2.4 Check all three

```
git --version
gh --version
python -c "import numpy; print(numpy.__version__)"
```

(On Windows the last line starts with `py`; on macOS and Linux with `python3`.) Three version numbers means you are ready. An error names the tool that is missing: go back to its step.

2.5 Your first NumPy check

Do this on paper first. Let $z = 3 + 4i$ and $w = 1 - 2i$. Work out $z + w$, zw , the conjugate z^* , and the modulus $|z|$. Then check each answer. In Python the imaginary unit is written `1j`, with no space, because `i` and `j` are ordinary variable names.

Save these lines in a file called `first_check.py` and run `python first_check.py`:

```
import numpy as np

z = 3 + 4j
w = 1 - 2j
print(z + w)      # (4+2j)
print(z * w)      # (11-2j)
print(np.conj(z)) # (3-4j)
print(abs(z))     # 5.0
```

If a printed value disagrees with your paper, the paper is almost always the one that is wrong. Find the step where it went wrong; copying the printed answer across teaches you nothing. That habit is what the NumPy checks in Parts A and B are for.

PITFALL · `2j` IS A NUMBER, `2*j` IS A VARIABLE

`2j` means $2i$. `2*j` means "2 times whatever is stored in the variable `j`", which is an error if there is none, and a silent wrong answer if there is one left over from a loop.

Part 3: A GitHub account and an SSH key

3.1 Create the account

Go to **github.com** and sign up. Use a username you are willing to show an employer: it becomes part of your professional record. Add your **college email** to the account (Settings, Emails). Classroom 50 sends its invitation there, and the same address later gets you the free GitHub Student Developer Pack.

3.2 Make an SSH key

An SSH key is a pair of files. The **private** key stays on your laptop. The **public** key you give to GitHub, which then recognises your laptop without a password. In your terminal (Git Bash on Windows):

```
ssh-keygen -t ed25519 -C "your.college.email@cb.students.amrita.edu"
```

Press Enter three times to accept the default location and an empty passphrase.

PITFALL · NEVER SHARE THE PRIVATE KEY

`~/.ssh/id_ed25519` is private. `~/.ssh/id_ed25519.pub` ends in `.pub` and is the one you give to GitHub. If you ever paste the private key into a chat, a website or a repository, delete the key from GitHub and make a new one.

3.3 Give the public key to GitHub

Print it:

```
cat ~/.ssh/id_ed25519.pub
```

You get one line starting `ssh-ed25519 AAAA...`. Copy the whole line. On GitHub, click your avatar at the top right, then **Settings, SSH and GPG keys, New SSH key**. Give it a title such as `laptop`, paste the line, and click **Add SSH key**.

3.4 Check it works

```
ssh -T git@github.com
```

The first time it asks whether to continue connecting: type `yes`. You should see:

```
Hi <your-username>! You've successfully authenticated, but GitHub does not
provide shell access.
```

That message is a success, including the part about shell access.

PITFALL · IF IT SAYS PERMISSION DENIED (PUBLICKEY)

Three usual causes, most likely first: you pasted the private key in place of the `.pub` one; you copied only part of the line; or the key is not loaded. For the last case:

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

3.5 Tell Git who you are

Git writes your name and email into every commit. Use your real name and your college email:

```
git config --global user.name "Your Full Name"
git config --global user.email "your.college.email@cb.students.amrita.edu"
```


Part 4: Join Classroom 50

Classroom 50 is the School's system for graded work. Each assignment is a private repository created for you the moment you accept it, under the `Amrita-School-of-AI` organisation. A grader runs on every submission and posts a report on your repository within a couple of minutes.

1. **Accept the invitation.** You will receive an email at your college address inviting you to the `Amrita-School-of-AI` organisation on GitHub. Sign in to GitHub with the account you made in Part 3 first, then open the link in the email and accept.
2. **Install the Classroom 50 tool and log in:**

```
gh extension install foundation50/gh-student
gh student login          # opens the browser once
gh student whoami        # prints your GitHub username
```

`gh student login` shows a short code and opens GitHub in your browser. Paste the code, approve, and return to the terminal.

PITFALL · USE ONE GITHUB ACCOUNT

The account that accepts the invitation is the account on the class roster. If `gh student whoami` prints a different username, run `gh auth logout`, then `gh student login` again with the right account.

Part 5: The A0 warm-up

A0 carries no marks. It exists so you meet Git, Classroom 50 and the grader's report on something that does not count. Do it now; every later part uses exactly the same steps.

5.1 Accept and clone

```
gh student accept Amrita-School-of-AI qts-mpqt a0-git-warmup
```

This creates your repository, `qts-mpqt-a0-git-warmup-<your-username>`, and prints its name. Download it to your laptop and go into it:

```
gh repo clone Amrita-School-of-AI/qts-mpqt-a0-git-warmup-<your-username>
cd qts-mpqt-a0-git-warmup-<your-username>
```

5.2 Read the README, do the tasks

The repository's `README.md` is the full statement. It lists each task and what the grader checks. Open the folder in any editor (VS Code, Notepad++, or a plain text editor) and work through it.

5.3 Submit

```
git add -A
git commit -m "Complete the warm-up"
gh student submit
```

`git add` chooses what goes into the next snapshot, `git commit` takes the snapshot with a message saying what changed, and `gh student submit` sends it to GitHub and tells the grader to run.

5.4 Read the report

About a minute later, on your repository's page on GitHub:

- a green tick or a red cross appears next to your latest commit;
- a **Release** appears, whose text is the grader's report: one line per check, PASS or FAIL, and for every failure, the reason.

Fix what the report names, then submit again. You may submit as many times as you like before the deadline; the last submission counts. Before the second and every later submission, run this first, because the submission itself added a commit on GitHub:

```
git pull --rebase origin main
```

NOTE · TWO FILES ARE NOT YOURS

`.gitignore` and the `.github/` folder belong to the course. Every submission puts them back the way they were, so editing them has no effect.

Part 6: The Qubit Workbook on Classroom 50

The statement for all three parts is `assignments/class-assignment.pdf` in the course repository. Each part is its own Classroom 50 item:

PART	ACCEPT WITH	FILES YOU ADD	HOW IT IS MARKED
A, complex numbers	<code>gh student accept Amrita-School-of-AI qts-mpqt mpqt-a</code>	<code>partA.pdf</code> , <code>checks_a.py</code>	10 marks, by hand
B, linear algebra	<code>gh student accept Amrita-School-of-AI qts-mpqt mpqt-b</code>	<code>partB.pdf</code> , <code>checks_b.py</code>	14 marks, by hand
C, qubit simulator	<code>gh student accept Amrita-School-of-AI qts-mpqt mpqt-c</code>	<code>qubit.py</code> , <code>report.pdf</code>	14 by the grader, 6 for the report by hand

The deadline of each part is shown on Classroom 50 and announced in class. The viva (6 marks) happens in class after Part C closes.

6.1 Parts A and B: handwritten work, scanned

1. Do the problems by hand, every step shown, in a notebook or on sheets.
2. Scan all the pages into **one** PDF. Google Drive's **Scan** button and Microsoft Lens both work: they crop the edges, fix the contrast and join the pages. Keep it under 25 MB; a scan of ten pages is normally 2 to 5 MB.
3. Copy the PDF into your Part A repository folder and name it exactly `partA.pdf` (`partB.pdf` for Part B).
4. Your repository already contains `checks_a.py` (`checks_b.py`), with an example of the format at the top. Add your NumPy checks to it, one `print` per answer, each line starting with the problem it checks, such as `A4(b):` . Check at least three answers in Part A and four in Part B. Run it with `python checks_a.py` ; it must run to the end without an error.
5. Submit:

```
git add partA.pdf checks_a.py
git commit -m "Part A"
gh student submit
```

For Parts A and B the grader only confirms that the PDF opens and that your checks file runs and prints enough labelled lines. The marks are given by hand after the deadline, from the PDF.

PITFALL · TEN PHOTOGRAPHS ARE NOT A SCAN

Separate photographs taken at an angle, in a shadow, are hard to read and easy to lose. Illegible working earns no method marks. Submit one scanned PDF, pages in order.

6.2 Part C: the qubit simulator

Accept `mpqt-c` and clone it as in 5.1. The repository holds `qubit.py`, with every function named and documented and an empty body, and the public tests in `tests/`. Fill in the functions as Unit 3 is taught. Check a stage at a time:

```
python tests/public.py qubit.py --stage states
```

The stages are `states`, `probabilities`, `gates`, `measure`, `tensor` and `expectation`. Each test prints `ok`, or `FAIL:` with what was expected and what your code gave. The repository's `README.md` also shows a one-line command that runs all six stages at once.

Write the report in any word processor and export it as `report.pdf` into the same folder. Submit both files the same way as in 6.1. The grader runs the public tests and extra hidden tests on the same functions; the release tells you your mark out of 14 straight away.

Part 7: Using Google Colab for the Python work

Colab runs Python in your browser with NumPy already installed. You can do all of the Python in this course there. Submitting still happens from your laptop's terminal with Git and `gh`, set up in Parts 2, 3 and 4, so install those even if you use Colab.

7.1 Start

Go to `colab.research.google.com`, sign in with your college Google account, and choose **New notebook**. Type code into a cell and press **Shift + Enter** to run it. The check in 2.5 works exactly as written.

7.2 Write a checks file for Part A or B

Copy the contents of `checks_a.py` from your Part A repository (open it on GitHub) into one cell, add your checks, and put `%%writefile checks_a.py` as the cell's first line:

```
%%writefile checks_a.py
import numpy as np

z1 = 3 + 2j
z2 = 1 - 4j
print("A1(c): z1 z2 =", z1 * z2)      # compare with your hand answer
```

Running the cell saves the file in Colab. Run it with a second cell containing `!python checks_a.py`. To get the file onto your laptop, open the **Files** panel (the folder icon on the left), right-click `checks_a.py`, and choose **Download**. Move it into your Part A repository folder, then submit as in 6.1.

7.3 Part C in Colab

Run these in a cell to fetch the starter and the public tests:

```
!git clone -q https://github.com/Amrita-School-of-AI/mpqt-25aid432
%cd mpqt-25aid432/exercises/mpqt-ex01-qubit-simulator
!cp starter/qubit.py qubit.py
```

Open `qubit.py` from the Files panel (double-click it) to edit it in Colab's editor. Test a stage with:

```
!python tests/public.py qubit.py --stage states
```

When you are ready to submit, download `qubit.py` from the Files panel, copy it over the `qubit.py` in your `mpqt-c` repository on your laptop, and submit from the terminal as usual.

PITFALL · COLAB FORGETS YOUR FILES

A Colab session's files are deleted when the session ends, after a period idle or when you close the tab.

Download `qubit.py` at the end of every sitting, or keep a copy in Google Drive. Submitting often also keeps your work safe: every submission is stored on GitHub.

Part 8: Marks, rules and getting help

How the marks work

COMPONENT	WEIGHT	WHAT IT IS
Class assignment	50%	The Qubit Workbook: Part A (10), Part B (14), Part C (20), viva (6)
End-semester examination	30%	Written paper on the whole syllabus
Class examination	20%	Multiple-choice test on Evolveus, Units 1 and 2

The full scheme is in the course handbook.

Rules

Late work. Marked out of 80 percent within 48 hours of the deadline; not accepted after that except with documented cause.

Your own work. Discuss approaches with anyone. Write your own answers and your own code.

AI tools. Permitted for learning: an explanation, another worked example, a hint. In the viva you will be asked to explain any step of Parts A and B and any line of Part C. What you cannot explain earns no marks.

When something goes wrong

Bring three things: what you expected, what you got, and what you have already tried. For Python or Git, bring the exact command and the exact message, copied as text. For a calculation, bring the page of working.

Things that catch nearly everyone

MESSAGE OR SYMPTOM	CAUSE AND FIX
<code>Permission denied (publickey)</code>	The SSH key is not set up. Redo 3.2 to 3.4.
<code>rejected ... (fetch first)</code> when submitting	GitHub has a commit you do not. Run <code>git pull --rebase origin main</code> , then submit again.
<code>nothing to commit</code>	You did not save the file, or it is in a different folder. Run <code>git status</code> to see what Git sees.
The grader cannot find <code>partA.pdf</code>	The file is named <code>PartA.pdf</code> , <code>partA.pdf.pdf</code> or similar. The name must be exact.
<code>ModuleNotFoundError: No module named 'numpy'</code>	NumPy is installed for a different Python. Use <code>py -m pip install numpy</code> on Windows, <code>python3 -m pip install numpy</code> elsewhere.
<code>gh student</code> says you are not on the roster	You are logged in with a different GitHub account. See the pitfall in Part 4.